README

CodeGraph integration for HelixCode

This directory incorporates **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`) into HelixCode as a **pinned, vendored third-party dependency** — not a git submodule.

See the full incorporation plan: `docs/research/codegraph/incorporation-plan.md`.

What CodeGraph is

CodeGraph is a pre-indexed **code knowledge-graph engine** for AI coding agents. It parses a codebase into a local graph of symbols (functions, classes, methods) and edges (calls, imports, inheritance), stores that graph in a local SQLite database (`<project>/.codegraph/codegraph.db`), and serves it to AI agents over the **Model Context Protocol (MCP)** — so an agent queries the graph instantly instead of spawning grep / glob / Read scans.

- Language: TypeScript, compiled to `dist/` JavaScript.
- Runtime: Node.js `>=18 <25` (host has `v22.19.0` — compatible).
- Parsing: `web-tree-sitter` (WASM tree-sitter — no native compile needed).
- Storage: SQLite + FTS5 index.
- License: MIT.
- Indexes 19+ languages including **Go** (HelixCode's primary language).

How HelixCode uses it

CodeGraph runs as an **MCP stdio server** (`codegraph serve --mcp`) that five CLI coding agents — Claude Code, OpenCode, Kimi CLI, Crush, Qwen Code — query for code-graph data. Agents call eight `codegraph_*` MCP tools (`codegraph_search`, `codegraph_context`,

`codegraph_callers`, `codegraph_callees`, `codegraph_impact`, `codegraph_node`, `codegraph_files`, `codegraph_status`) instead of brute-force file scans.

Layout

```
tools/codegraph/
├── README.md           # this file
├── codegraph.version   # pinned upstream version (literal string)
├── install.sh          # idempotent installer (CG3)
├── verify.sh           # anti-bluff end-to-end proof (filled in
Phase C / CG10)
└── agents/             # per-agent MCP registration helpers
(Phase B / CG5-CG9)
```

- The CodeGraph runtime installs into `tools/codegraph/node_modules/` via `npm install --prefix tools/codegraph` — gitignored (CONST-053).
- The scanned-graph artefact `.codegraph/` is created at the HelixCode repo root and inside `helix_code/` — gitignored (fully recreatable by `codegraph init -i`).
- `codegraph.version` and the `.sh` scripts ARE versioned (generator/source).

Usage

```
# 1. Install the pinned CodeGraph runtime (idempotent).
tools/codegraph/install.sh

# 2. Initialize + scan (done by CG4; re-runnable any time).
tools/codegraph/node_modules/.bin/codegraph init -i        # repo
        root
cd helix_code && ../tools/codegraph/node_modules/.bin/codegraph
        init -i

# 3. Inspect the graph.
tools/codegraph/node_modules/.bin/codegraph status .       # JSON
        node/edge counts
tools/codegraph/node_modules/.bin/codegraph query Provider # real
        symbols

# 4. Verify end-to-end (Phase C).
tools/codegraph/verify.sh
```

Per-agent registration

CodeGraph is registered with each of the five HelixCode CLI agents as an MCP **stdio** server — the same process (`codegraph serve --mcp`) for all five; only the config file location and JSON/TOML shape differ. The `agents/` directory holds one idempotent registration helper per agent; run the one(s) you need:

```
tools/codegraph/agents/register-claude.sh    # → .mcp.json +
        mcp__codegraph__* permission
tools/codegraph/agents/register-opencode.sh  # → opencode.jsonc
        (mcp.codegraph, command[] array)
tools/codegraph/agents/register-kimi.sh      # → ~/.kimi/mcp.json
        (mcpServers shape)
tools/codegraph/agents/register-crush.sh     # → .crush.json
        (mcp.codegraph, type stdio)
tools/codegraph/agents/register-qwen.sh      # → .qwen/
        settings.json (mcpServers shape)
```

Agent	Config file (project scope)	Wrapper key	command form
Claude Code	`.mcp.json`	`mcpServers`	command + args
OpenCode	`opencode.jsonc`	`mcp`	`command[]` array
Kimi CLI	`~/.kimi/mcp.json`	`mcpServers`	command + args
Crush	`.crush.json`	`mcp`	command + args
Qwen Code	`.qwen/settings.json`	`mcpServers`	command + args

After registration, each agent exposes the eight `codegraph_*` tools. Verify the agent actually reaches CodeGraph (not just that the config file exists) with the agent's own MCP-list command — e.g. `claude mcp list`, `qwen mcp` — and confirm `codegraph` appears with its tools. A config file that merely *contains* the entry is NOT proof; the PASS bar is the agent invoking a `codegraph_*` tool and returning real graph data (CONST-035).

Anti-bluff contract (CONST-035)

"Installed" is not "working". Every PASS for CodeGraph carries captured runtime evidence: a real `codegraph status` JSON with non-zero `nodes` / `edges` / `files`. A zero-node graph is a FAIL. Evidence captured during incorporation lives under `docs/research/codegraph/evidence/`.

Version pin

The pinned version is the single literal string in `codegraph.version`. Bumping it is an explicit, reviewed change: edit `codegraph.version`, re-run `install.sh`, re-run `verify.sh` and every `CG-CHALLENGE-*`.